

TKOM 2026L - Dokumentacja wstępna, Mateusz Jakubiak

Opis funkcjonalności oraz przykłady kodu

Język jest zgodnie z opisem na USOSie:

- Statycznie typowany - typ zmiennej ustalony przy deklaracji nie może na późniejszym etapie ulec zmianie, co ma wpływ na sposób odczytywania wartości zmiennych z tablicy symboli - może być jej odczytywanie szybsze niż dla języka dynamicznie typowanego.
- Silnie typowany, tj. konwersje muszą być jawne, dajmy na to: `3 + int("4")` daje 7 (jako `int`, liczbę).
- Zmienne domyślnie niemutowalne, podobnie jak w Rustcie.
- Wbudowany typ kolekcji homogenicznych, tzn. przechowujących elementy jednego typu, jak `std::vector` w C++
- Liczby całkowite mogą zawierać separatory napisów;

Typy wbudowane w języku

`int`
`uint`
`float`
`bool`
`char`
`string`
`list<T>`

`string` jest traktowany semantycznie jako `list<char>` w celu umożliwienia używania operatorów zdefiniowanych dla `list` na zmiennych `string` w projektowanym języku programowania.

Typ `int` ma zakres od **-2 147 483 648** do **2 147 483 647** i dochodzi do overflow przy wykonywaniu operacji arytmetycznych

Typ `uint` ma zakres od **0** do **4 294 967 295** i również dochodzi tu do overflow, zapewne rozsądne byłoby zgłoszenie ostrzeżenia (warning) gdy w danej operacji zachodzi overflow.

`Float` analogiczny do `double` w C++, tj. liczba zmiennie przecinkowa podwójnej precyzji.

Tworzenie własnych klas

```
class User {
    name: string
    age: int
    isActive: bool
    country: string

    fun printName() -> void {
        print(name)
    }

    fun isAdult() -> bool {
        return age >= 18
    }

    mut fun setName(name: string) -> void {
        this.name = name
        // Jeśli spróbujemy zrobić name = name, to chyba nie powinno się
        // stać nic - name jako atrybut User jest przykryte przez argument
        // metody setName, więc jest to analogiczne jak poniżej:
        // int x = 2
        // x = x
    }

    mut fun birthday() -> void {
        this.age = this.age + 1
    }
}

class Order {
    customer: User
    price: float
    id: int
}
```

Jak widać wyżej, wewnątrz klas można definiować funkcję - określimy to mianem metod tych klas. Przed nimi może być słowo kluczowe 'mut' - oznacza ono, że dana metoda może być wywołana tylko dla mutowalnych zmiennych (obiektów klas jak User czy Order). Oczywiście pewnie dałoby się obejść bez tego - wewnątrz tabeli symboli przechowuje się fakt, czy dana zmienna user (obiekt User) jest mutowalna czy nie (tzn. czy została zadeklarowana jako **mut User userMutable = User{...}** czy po prostu jako **User userNonMutable = User{...}**), i wtedy przy wywołaniu **userNonMutable.birthday()** podejmowana jest próba modyfikacji atrybutu age, i jeśli **userNonMutable** jest stałą (co w projektowanym języku jest domyślne), to zgłaszany jest błąd semantyczny. Oczywiście może dojść do sytuacji, gdzie do takiego

stanu rzeczy doprowadzi cała kaskada wywołań, i wtedy tę kaskadę trzeba będzie pewnie wypisać przy okazji zgłaszania użytkownikowi tego błędu. Przyjęcie natomiast istnienia czegoś takiego jak **mut fun setName...** wewnątrz definicji typu User umożliwi potencjalnie jeszcze przed próbą interpretacji programu napisanego w projektowanym języku odkrycie, że gdy byśmy mieli **fun setName(name) {this.name = name}** to próbujemy zmodyfikować atrybut obiektu klasy User wewnątrz metody bez klasyfikatora mut, co jest błędem semantycznym statycznym.

Zakłada się możliwość definiowania metody compare:

```
class User {
    name: string
    age: int

    fun compare(other: User) -> int {
        // < 0 jeśli this < other
        // = 0 jeśli this == other
        // > 0 jeśli this > other
    }
}
```

Przyjmuje ona jako jedyny argument other: T (jeśli jest definiowana dla class T), i zwraca int. Używana jest do porównywania klas zdefiniowanych przez użytkownika, tzn. do operatorów <, <=, >, >=, ==, !=. W przypadku braku zdefiniowania compare i próbie porównania obiektów klas dla której metoda ta nie jest zdefiniowana jest zgłaszany błąd semantyczny podczas statycznej analizy semantycznej.

Tworzenie obiektów klas

```
User nonMutUser = User{name: "Anna", age: 25, isActive: true, country: "PL"}
User.nonExistent           // Błąd semantyczny statyczny (odwołanie
                           // do nieistniejącego pola
```

```
Order nonMutOrder =
    Order{
        customer: User{name: "Kamil", age: 20, isActive: true, country: "PL"},
        price: 20.1,
        id: 1
    }
```

```
mut Order mutOrder = nonMutOrder
// Powyżej mutOrder tworzone jest jako głęboka kopia nonMutOrder
```

```
nonMutOrder.customer.setName("AnnaChange") // Błąd semantyczny
```

```
// Wynika to stąd, że mutowalność jest głęboka, to znaczy jeśli w skład Order
// wchodzi atrybut klasy User, to gdy Order nie jest mut, tak samo składowa
```

```
// User (czyli customer w Order) też nie jest mut
```

```
mutOrder.customer.setName("AnnaChange") // Okej
```

Zaproponowane tworzenie obiektów przez głębokie kopiowanie wydaje mi się bardziej zgodne z tym, że projektowany język ma w wymaganiach przekazywanie zmiennych przez wartość.

Możliwe jest także oczywiście przypisywanie pod customer "nie w locie":

```
User userAnna = User{name: "Anna", age: 25, isActive: true, country: "PL"}
Order orderAnna =
    Order{customer: userAnna, price: 20.1, id: 1}
```

Tworzenie list obiektów w locie i dostęp do pól obiektów

```
User userAnna = User{name: "Anna", age: 25, isActive: true, country: "PL"}
```

```
list<User> users = [
    User{name: "Bob", age: 31, isActive: false, country: "DE"},
    userAnna,
    User{name: "Ewa", age: 22, isActive: true, country: "PL"}
]
```

```
users[0].name // To zwróci "Anna"
order.customer.age
```

Statyczne typowanie i mutowalność

```
int a = 5
a = 7 // błąd semantyczny statyczny
a = "tekst" // błąd typu (semantyczny statyczny)
```

```
mut list<User> users = [
    User{name: "Anna", age: 25, isActive: true, country: "PL"},
    User{name: "Bob", age: 31, isActive: false, country: "DE"},
    User{name: "Ewa", age: 22, isActive: true, country: "PL"}
]
```

```
users[0].name = "Kasia"
// Teraz mamy users = [
//     User{name: "Kasia", age: 25, isActive: true, country: "PL"},
//     User{name: "Bob", age: 31, isActive: false, country: "DE"},
//     User{name: "Ewa", age: 22, isActive: true, country: "PL"}
// ]
```

```
mut int a = 5
a = 7 // poprawne
```

```

a = "tekst" // Błąd semantyczny statyczny
list<float> xs = [2.5, "a"] // Błąd(1) semantyczny statyczny

int a1, b1, c1 = 5
mut int a2, b2, c2 = 10

mut int a3, b3, c3 // Domyślna wartość dla int to 0
int a4, b4, c4 // Błąd (na poziomie gramatyki, czy semantyczny?)

```

Silne typowanie i konwersje między typami

```

int a = 5
string s = "abc"
int x = a + s // Błąd(2) semantyczny statyczny
list<float> xs = [2.5, 5 as float, 3.1]
list<float> xs1 = [2.5, 5, 3.1] //Błąd(3) semantyczny statyczny, brak konwersji int a float

```

W powyższym przykładzie można Błąd(1) rozumieć jako brak zdefiniowania operatora + dla zmiennych int oraz string (w sensie int + string), natomiast Błąd(2) i Błąd(3) to błędy semantyczne wynikające ze wrzucenia do list<float> odpowiednio string lub int.

- **bool as int** - gdy false to na 0, gdy true to na 1
- **int as bool** - dla $x > 0$ daje true, dla $x \leq 0$ daje false
- **bool as uint** - dla false daje uint 0, dla true daje uint 1
- **uint as bool** - dla 0 daje false, dla dodatniej liczby true
- **int as uint** - jeśli int w zakresie uint konwersja poprawna, inaczej błąd semantyczny (statyczny? dynamiczny?) lub warning z zawinięciem i dalsze wykonanie
- **uint as int** - podobnie jak powyżej, błąd semantyczny lub warning z zawinięciem gdy uint ujemne (poza zakresem int)
- **int as float, uint as float**
- **float as int** -3.9 as int daje -3
- **float as bool** - $x > 0.0$ daje true, $x \leq 0.0$ daje false
- **list<char> as string, string as list<char>**
być może można traktować te typy jako równoważne

Zakładam wysoki priorytet dla operatora as. Dyskusyjna jest kwestia, czy język powinien (mimo silnego typowania) dopuszczać operacje arytmetyczne mieszane, jak $5.0 / 2$, i jaki typ powinien wówczas zwracać. Dla prostoty użytkowania zapewne założę, że działania arytmetyczne na mieszanych zmiennych (int i float) będą zwracać float, tzn. dokonywana będzie niejawna konwersja. Operacje arytmetyczne na int/float (lub float/int) są jedynym przypadkiem w języku, kiedy dokonywana jest niejawna konwersja.

```

float x = 2.0 / 5 // teraz x wynosi 0.4, tj. dokonywana jest niejawna
                  // konwersja 2.0 as int / 5

float y = 2.0 as int / 5 // 2 / 5 daje nam 0, typ int, a przy przypisywaniu
                          // nie występuje już niejawna konwersja, stąd
                          // mamy tutaj błąd semantyczny statyczny

```

```

int z = (4.0 / 3.0) as int      // teraz z wynosi 1, wymagane nawiasy ze względu
                                // na priorytet operatora as

list<int> xs = list_of_floats as int
                                // W powyższym przypadku as int wykonywane dla
                                // każdego elementu list_of_floats

bool x_true = true
int a = x_true as int + 7      // Teraz a wynosi 1 + 7 = 8
int a1 = x_true + 7            // Błąd semantyczny statyczny, brak zdefiniowania operatora
                                // + dla typów (bool, int)

float f1 = x_true as int + 7.0
                                // Teraz f1 wynosi 8.0 - jak wspomniano, operatory
                                // arytmetyczne dokonują niejawnych konwersji
                                // dla par mieszanych int/float lub float/int

```

Dodawanie i usuwanie elementu do kolekcji

```

mut list<int> xs = [1, 2, 3]
xs.push(4)
xs.push([1, 2, 5]).push(5)
// Teraz xs wynosi [1, 2, 3, 1, 2, 5, 5]

xs.push("a") // Błąd semantyczny statyczny

mut list<User> users = [
  User{name: "Anna", age: 25, isActive: true, country: "PL"},
  User{name: "Bob", age: 31, isActive: false, country: "DE"},
  User{name: "Ewa", age: 22, isActive: true, country: "PL"}
]

users.push( User{name: "Marysia", age: 26, isActive: true, country: "PL"} )

list<int> xs = [1, 2, 3]
list <int> xs1 = xs.push(4)      // xs1 wynosi [1, 2, 3, 4]
mut list<int> xs2 = xs1.push(5)  // xs2 wynosi [1, 2, 3, 4, 5]

xs2.push(8)                    // Błąd semantyczny, xs1 nie jest mutowalne

xs2.insert(0, -3)               // Teraz xs2 wynosi [-3, 1, 2, 3, 8]

```

Metoda wbudowana kolekcji push modyfikuje listę na której dokonywana jest ta operacja. Przy przypisywaniu `list<int> xs2 = xs1.push(2)` dokonywane jest kopiowanie `xs1` (nie referencja), zgodnie z wymaganiami. Metoda `insert(int index, T elem)` wstawia element do `list<T>` przed indeksem `index`. Metody `push` i `insert` kopią wstawiane elementy, to znaczy

gdy zrobimy `listOfUsers.push(user)`, a w następnej linii zrobimy np. `user.name = "changed name"`, to obiekt na końcu listy `listOfUsers` nie ulegnie zmianie i będziemy mieć `listOfUsers[size listOfUsers - 1] == "old name"`.

```
int x = xs.pop()
string s = xs.pop()           // Błąd semantyczny statyczny
xs.pop()                      // Usuwa ostatni element z listy
mut list<int> xs3 = [5, 7, 10, 12]
xs3.remove(1)                 // Teraz xs3 wynosi [5, 10, 12]
int xs3at1 = xs.remove(1)     // xs3at1 wynosi 10
```

Metoda `pop()` modyfikuje listę na której jest dokonywana i zwraca ona ten ostatni element (czyli jest funkcją `(list<T>) -> T`). Metoda `remove(int index)` usuwa element o podanym indeksie i zwraca go.

Ekstrakcja elementu z kolekcji

```
list<int> xs = [10, 20, 30]
int a = xs[1]                 // Teraz a wynosi 20
int b = xs[3]                 // Błąd semantyczny (dynamiczny?)
list< list<int> > list_of_lists = [ [1, 2], [2, 3, 4], [2, 5, 6] ]
print(list_of_lists[0])       // Wypisze [1, 2]
print(list_of_lists[2][1])    // Wypisze 5

list[1:]                      // lista bez pierwszego elementu (list[0])
list[5:8]                     // lista z elementów list[5], list[6], list[7]
```

Operacja 1 na listach - konkatencja (operator +)

```
list<int> a = [1, 2]
list<int> b = [3, 4]
mut list<int> c = a + b       // Teraz c wynosi [1, 2, 3, 4]
c = a.push(10) + b           // Teraz c wynosi [1, 2, 10, 3, 4] oraz a wynosi [1, 2, 10]
list<int> d = a + [1, 2, 20]  // d wynosi [1, 2, 1, 2, 20]
```

Operacja 2 na listach - odejmowanie (operator -)

```
list<int> a = [1, 2, 3, 2]
list<int> b = [2, 5]
list<int> c = a - b           // Teraz c wynosi [1, 2, 3]
list<int> d = a - b - [3]     // = (a - b) - [3]      d wynosi [1, 2]
[1, 2, 3] - ["jeden"]        // Błąd semantyczny
```

Operacja odejmowania na listach odejmuje przy $a - b$ taką liczbę elementów z a , jakie są w b (jeśli w a pewne elementy się duplikują).

Operacja 3 na listach - przecięcie kolekcji (operator $*$)

```
list<int> a = [1, 2, 3, 2]
list<int> b = [2, 2, 5]
list<int> c = a * b // Teraz c wynosi [2, 2]
list<int> d = a * b * [2] // = (a * b) * [2] // Teraz d wynosi [2]
[1, 2, 3] * ["jeden"] // Błąd semantyczny
```

Operacja przecięcia kolekcji zwraca nową listę zawierającą elementy wspólne dla obu kolekcji. Liczba wystąpień danego elementu w wyniku jest równa minimum liczby jego wystąpień w obu kolekcjach.

Kolejność elementów w wyniku odpowiada kolejności ich występowania w lewej kolekcji.

Operacja 4 na listach - grupowanie kolekcji (operator $\%$)

```
class Group {
    key: string
    value: int
    isActive: bool
}

list<Group> groups = [
    Group{key: "A", value: 10, isActive: true},
    Group{key: "B", value: 20, isActive: false},
    Group{key: "A", value: 30, isActive: true},
    Group{key: "C", value: 40, isActive: true},
    Group{key: "B", value: 50, isActive: true}
]

list<list<Group>> grouped = groups % key
```

```
[
    [
        Group{key: "A", value: 10, isActive: true},
        Group{key: "A", value: 30, isActive: true}
    ],
    [
        Group{key: "B", value: 20, isActive: false},
        Group{key: "B", value: 50, isActive: true}
    ],
    [
        Group{key: "C", value: 40, isActive: true}
    ]
]
```



```
]
]
```

Operator `%` grupuje elementy kolekcji według wartości wyrażenia podanego po jego prawej stronie. Jeśli `xs` jest typu `list<T>`, a wyrażenie po prawej stronie operatora ma typ `K` i jest obliczane w kontekście elementu typu `T`, to wynik operatora `%` ma typ `list<list<T>>`. Każda lista wewnętrzna zawiera elementy, dla których wartość wyrażenia grupującego jest taka sama.

Dzięki takiemu rozwiązaniu nie trzeba wprowadzać do języka wbudowanego słownika.

Operacja 5 na listach - filtrowanie kolekcji (operator `?`)

```
users ? isActive
```

Jeśli `users` jest typu `list<User>`, to zostanie zwrócone `list<User>` złożone z tylko tych obiektów `User`, które mają pole `isActive` ustawione na `true`.

```
users ? isActive && name == "Anna"
```

Zwróci aktywnych użytkowników o imieniu Anna.

```
fun isActiveAndNameAnna(User: user) -> bool {
    return user.isActive && user.name == "Anna"
}
user ? isActiveAndNameAnna
```

W przypadku filtrowania, jeśli użyjemy np. `isActiveAndNameAnna` dla typu `User`, który przecież nie ma takiego pola zdefiniowanego, interpreter dokona próby dopasowania identyfikatora do funkcji, i dopiero jeśli nie ma takiej funkcji, to zostanie zgłoszony błąd semantyczny (statyczny).

Ponadto zakładana jest poniższa funkcjonalność:

```
fun isActiveAndNames(User: user, string: name) -> bool {
    return user.isActive && user.name == "Anna"
}

user ? isActiveAndNames("Anna")
```

To będzie działać analogicznie do powyższego przykładu z `isActiveAndNameAnna` - gdy funkcja podawana do parametru `?` filtrowania będzie miała więcej niż jeden argument, to przy takim wywołaniu zostaną przekazane jako drugi, trzeci... argument.

Operacja 6 na listach - mapowanie kolekcji (operator `|>`)

```
fun square(int: x) -> bool {
```

```
    return x*x
}
```

```
fun toThePowerOf(int: x, int: y) -> int {
    return x^y
}
```

```
list<int> xs = [1, 2, 3]
list<int> ys = xs |> square           // teraz ys wynosi [1, 4, 9]
list<int> ys = xs |> toThePowerOf(3)  // teraz ys wynosi [1, 8, 27]
list<string> zs = xs |> int_to_string // teraz zs to ["jeden", "dwa", "trzy"]
list<int> zs1 = xs |> int_to_string   //Błąd semantyczny statyczny
```

//Analogicznie oczywiście

```
list<string> namesOfUsers = users |> name
list<string> namesOfActiveUsers = users ? isActive |> name
```

fun int_to_string(x: int) -> string {return ""...}, czyli typ (int) -> string.

W przypadku ogólnym map może zmieniać typ elementów, więc jeśli **fun1: T -> (U)**, to **list<T> -> list<U>**.

Operacja 7 na listach - zawieranie się elementu w kolekcji (operator contains)

```
list<int> xs = [3, 5, 8]
bool a = xs contains 5           // true
bool b = xs contains 2           // false
bool c = xs contains [5, 8]      // true
bool d = xs contains [5, 5, 8]   // false
```

a contains b (a: list<T>, b: list<T>) zwraca bool true, gdy wszystkie elementy z b należą do a.

```
bool e = xs contains 5           // true
bool f = xs contains 10          // false
```

Operacja 8 na listach - licznosc kolekcji (operator count)

```
list<int> xs = [7, 8, 9]
int n = count xs                 // Teraz n jest rozmiarem kolekcji, czyli wynosi 3
int n1 = count users ? isActive // Zwraca liczbe aktywnych użytkowników
```

Operacja 9 na listach - odwrotność kolekcji (operator reverse)

```
list<int> xs = [1, 2, 3, 4]
list<int> ys = reverse xs
// Teraz ys wynosi [4, 3, 2, 1]
```

```
list<int> zs = reverse (xs - [2])
// zs = [4, 3, 1]
```

```
reverse 5 // Błąd semantyczny
```

Operator `reverse` zwraca nową kolekcję, w której kolejność elementów jest odwrócona, jest więc typu `list<T> -> list<T>`.

Operacja 10 na listach - spłaszczanie kolekcji (operator flatten)

```
list<list<int>> xs = [[1, 2], [3, 4], [5]]
list<int> ys = flatten xs
// Teraz ys wynosi [1, 2, 3, 4, 5]
```

```
list<list<string>> zs = [["a"], ["b", "c"]]
list<string> ws = flatten zs
// ws = ["a", "b", "c"]
```

```
flatten [1, 2, 3] // Błąd semantyczny
```

Operator `flatten` spłaszcza kolekcję o jeden poziom, zachowując kolejność elementów. Jest więc nieco jak funkcja typu `list<list<T>> -> list<T>`

Operacja 11 na listach - mapowanie i spłaszczanie kolekcji (operator flatten oraz |>)

```
flatten (xs |> f)
```

```
fun neighbors(x: int) -> list<int> {
  return [x - 1, x, x + 1]
}
```

```
list<int> xs = [2, 5]
list<int> ys = flatten (xs |> neighbors)
// xs |> neighbors = [[1, 2, 3], [4, 5, 6]]
// ys = [1, 2, 3, 4, 5, 6]
```

```
=====
```

```
fun chars(s: string) -> list<char> {
  return s as list<char>
}
```

```
list<string> words = ["ala", "kot"]
list<char> letters = flatten (words |> chars)
// words |> chars = [['a','l','a'], ['k','o','t']]
// letters = ['a','l','a','k','o','t']
```

```
=====
```

```
class User {
  name: string
  tags: list<string>
}
```

```
fun get_tags(u: User) -> list<string> {
  return u.tags
}
```

```
list<User> users = [
  User{name: "Anna", tags: ["admin", "active"]},
  User{name: "Bob", tags: ["guest"]},
  User{name: "Ewa", tags: ["active", "editor"]}
]
```

```
list<string> all_tags = flatten (users |> get_tags)
// all_tags = ["admin", "active", "guest", "active", "editor"]
```

Mapowanie i spłaszczanie nie wymaga osobnego operatora i może być wykonane przez kombinację flatten oraz |>.

Operacja 12 na listach - redukcja kolekcji (operator reduce with ... from ...)

```
fun add(acc: int, x: int) -> int {
  return acc + x
}
```

```
list<int> xs = [1, 2, 3, 4]
int s = xs reduce with add from 0
```

W przypadku tego operatora funkcja przekazywana do reduce with from ma postać:

```
xs : list<T>,
init : A,
f : (A, T) -> A
```

xs reduce with f from init

Definiowanie funkcji

```
fun nazwa(param1: Typ1, param2: Typ2) -> TypWyniku {  
    ...  
}
```

```
fun is_even(x: Int) -> Boolean {  
    return x % 2 == 0  
}
```

```
fun square(x: Int) -> Int {  
    return x * x  
}
```

```
fun add(acc: Int, x: Int) -> Int {  
    return acc + x  
}
```

```
fun join_step(acc: String, x: String) -> String {  
    if acc == "" {  
        return x  
    }  
    return acc + ", " + x  
}
```

```
fun join_strings(xs: List<String>) -> String {  
    return xs.reduce { acc, x, _> join_step(acc, x) }  
}
```

Pętla for

```
for Typ ident in expression {  
    ...  
}
```

Powyższa struktura pasuje do statycznego typowania (for int i in list_of_ints...)

Wyrażenie expression powinno dawać listę list<Typ>, w przeciwnym razie zgłaszany jest błąd semantyczny.

```
list<list<int>> list_of_lists_of_ints = [  
    [1, 2],  
    [3, 4, 5],  
    [6]  
]
```

```
for list<int> a in list_of_lists_of_ints {  
    print(a)  
}
```

```
// Wypisze po kolei listy
```

```
for User u in users ? isActive {  
    print(u.name)  
}
```

```
// Wypisze po kolei imiona aktywnych użytkowników
```

```
for string name in users |> name {  
    print(name)  
}
```

```
// users |> name daje nam list<string>
```

```
list<int> int_list = [1, 2, 3]  
for int i in int_list {  
    i = 10 // Błąd semantyczny statyczny  
}  
for int i in [] {  
    print(i)  
}
```

```
// Dobrze pytanie, co powinno się stać? Pusta lista może być niejednoznaczna semantycznie.
```

```
for int i in [7, 2] + [1, 3] {  
    print(i) // Wypisze po kolei: 7, 2, 1, 3  
}
```

Instrukcja if

```
int x = 0
```

```
if x > 0 {  
    print("dodatnia")  
} else if x < 0 {  
    print("ujemna")  
} else {  
    print("zero")  
}
```

Standardowy if (if expression {} else if expression {} else {}). Expression musi dawać bool.

Instrukcja while

```
mut int i = 0
```

```
while i < 5 {
    print(i)
    i = i + 1
}
```

// wypisuje po kolei 0, 1, 2, 3, 4

```
=====
```

```
list<string> words = ["ab", "haha", "def"]
mut int i = 0
```

```
while i < count words {
    print(words[i])
    i = i + 1
}
```

// Wypisze po kolei ab, haha, def

```
=====
```

```
mut int i = 5
```

```
while i {
    print(i)
    i = i - 1
}
```

// Błąd semantyczny (statyczny?) - expression w while expression {} musi mieć typ bool.
Ogólnie w języku statycznie typowanym jeszcze przed wykonaniem programu możliwe jest zorientowanie się zmienną jakiego typu jest dane wyrażenie.

Shadowing zmiennych i redeklaracja w wewnętrznych blokach

```
int x = 1
```

```
for int i in [10, 20] {
    int x = i
```

```
    if i > 15 {
        int x = i + 1
        mut int j = 0
```

```
        while j < 2 {
            int x = j
            print(x) // (a)
```

```

    j = j + 1
}

print(x) // (b)
}

print(x) // (c)
}

print(x) // (d)

```

W powyższym kodzie redefiniujemy zmienną x wewnątrz bloków, co powoduje przykrycie zmiennej x z bloku wyżej. Przy i = 10 w pierwszym wykonaniu for nie wykonana się w ogóle if i > 15 {} i z (c) wypisane zostanie **10**, następnie mamy i = 20, więc z int x = i mamy x = 10, ale wewnątrz while przykrywamy int x = j i z (a) wypisujemy **0**, następnie z (a) wypisujemy **1**, następnie z (b) wypisujemy **21**, następnie z (c) wypisujemy **20**, następnie na końcu z (d) wypisujemy **1**,

```

mut int x = 5
mut int i = 0

while i < 2 {
    i = i + 1
    x = 7
    print(x) // (a)
}

print(x) // (b)

```

// Tutaj z (a) wypisujemy 7, potem znowu 7, a następnie z (b) znowu 7. Nie //zadeklarowaliśmy nowej zmiennej x wewnątrz bloku (zamiast "int x = 7" // mamy po prostu "x = 7"), czyli zmodyfikowaliśmy x z bloku wyżej

Importowanie plików i uruchamianie programu .djm zapisanego w pliku

Każdy import modułu wprowadza przestrzeń nazw odpowiadającą nazwie importowanego pliku bez rozszerzenia .djm, jak w przykładzie użycia poniżej.

```

import * from "file1.djm"
import Class2, fun2, var2 from "file2.djm"

file1.Class1, file1.fun1(...), file1.var1
file2.Class2, file2.fun2(...), file2.var2

```


W kolejnym z kolei przykładzie pokazano, że można definiować funkcję o tej samej nazwie jak ta w importowanym pliku, i odwoływać się do tej w danej momencie wymaganej z użyciem przestrzeni nazw.

Ogólnie jednak użycie przestrzeni nazw w przedrostku jest zawsze wymagane, nawet gdy nie ma problemu z przesłanianiem, to znaczy nawet gdyby w poniższym przykładzie w pliku main.djm nie byłoby definicji fun2. Takie podejście może być uciążliwe dla użytkownika, mi jednak ułatwi nieco implementację w kwestii poszukiwania podczas analizy semantycznej konfliktu nazw, oraz braku konieczności wstrzykiwania do tablicy symboli funkcji / klas / stałych zdefiniowanych w importowanych plikach.

```
===== file2.djm =====
fun fun2() -> { print("file2") }
=====

===== main.djm =====
import fun2 from "file2.djm"
fun fun2() -> { print("main") }

fun main() -> {
    fun2()          // Ta linia wypisze "main"
    file2.fun2()    // Ta linia wypisze "file2"
}
=====
```

Zmienne globalne muszą być stałymi (deklarowane bez przedrostka mut).

Wykonanie programu zapisanego w pliku .djm będzie dokonywane z terminala jak poniżej

```
djm-interpreter ./main.djm
```

Wymagana jest obecność funkcji main w pliku main.djm (z powyższego przykładu). Jeśli jednak inne main() występuje w pliku file1.djm importowanym poprzez `import * from "file1.djm"` to niekoniecznie musi być to błędem, można zignorować jego obecność w importowanym pliku i instrukcje w nim tam zawarte.

```
fun main() -> void {
    print("Hello")
}
```

Funkcja main może przyjmować argumenty i zwracać wartość, wyświetlaną na końcu działania programu (main1.djm completed with result: xxx).

```
fun main(args: list<string>) -> int {
    print(args[0])
    return 0
}
```

```
djm-interpreter ./main.djm arg1 arg2 arg3
```

Analizator leksykalny

Słowa kluczowe:

class, fun, mut, if, while, for, return,
import, from, as, this,
reduce, with

Słowo "from" jest używane zarówno przy importowaniu, jak i przy operatorze (konstrukcji) reduce ... with ... from ..., jednakże oczywiście nie w gestii leksera jest żeby sprawdzać poprawne jego użycie, a jedynie żeby wykryć samo to słowo i odróżnić je od identyfikatora (nazwy zmiennej).

reduce with from jest właściwie konstrukcją składniową, nie operatorem, i w gestii parsera jest już rozpoznanie odpowiednich wyrażeń wewnątrz tej konstrukcji.

expr reduce with expr from expr
xs reduce with add from 0
(xs |> f) reduce with g from 10

Identyfikatory:

Zaczynają się od litery, mogą składać się z liter, cyfr oraz podkreślników. Używane do nazywania zmiennych, klas i funkcji.

Literały

Literały int (złożone z cyfr), float (cyfry rozdzielone kropką), tekstowe (wewnątrz cudzysłowu), znakowe (wewnątrz ' ').

Możliwe użycie separatorów przy zapisie int: 200'000

Operatory:

->
=, +, -, *, /, ^,
&&, ||, !=, ==, <=, >=, <, >
?, |>, %,
contains,
count, reverse, flatten

Powyższe count, reverse, flatten można traktować jako operatory unarne (działające na listach, ale znowu - co to obchodzi analizator leksykalny na czym one działają?), natomiast contains to operator binarny.

Delimitery:

(){}[], ::, .

Lekser winien działać, jak wielokrotnie było podkreślane podczas wykładów, leniwie, przez co rozumiem, że parser zgłasza do modułu leksera chęć otrzymania kolejnych tokenów, i dopiero wówczas lekser podejmuje próbę tych kolejnych tokenów otrzymania. Potencjalnie taki sposób współdziałania obydwu tych modułów może umożliwić przekazywanie kodu źródłowego .djm przez sieć, w pewnych fragmentach, co z kolei oczywiście wymaga pewnej abstrakcji źródła, strumienia, z którego kod źródłowy .djm będzie pochodzić.

Wzór struktury, w jakiej w C++ będzie przechowywany token może wyglądać jak poniżej:

```
struct SourceLocation {
    std::string file;
    int line;
    int column;
};

struct Token {
    TokenType type;
    std::string lexeme;
    SourceLocation loc;
};
```

Linia i kolumna oraz plik będą bardzo przydatne przy wypisywaniu błędów.

```
enum class TokenType {
    IDENT,
    INT_LITERAL, FLOAT_LITERAL, STRING_LITERAL, CHAR_LITERAL,

    KW_INT, KW_UINT, KW_FLOAT, KW_BOOL, KW_CHAR, KW_STRING, KW_VOID,
    KW_LIS,
    TKW_CLASS, KW_FUN, KW_MUT, KW_IF, KW_WHILE, KW_FOR, KW_RETURN,
    KW_IMPORT, KW_FROM, KW_AS, KW_THIS,
    KW_REDUCE, KW_WITH, KW_INT, KW_FLOAT, KW_STRING, KW_CHAR,

    OP_ASSIGN, OP_PLUS, OP_MINUS, OP_MUL, OP_DIV, OP_POW,
    OP_AND, OP_OR, OP_NEQ, OP_EQ, OP_LE, OP_GE, OP_LT, OP_GT,
    OP_FILTER, OP_PIPE, OP_GROUP,
    OP_CONTAINS, OP_COUNT, OP_REVERSE, OP_FLATTEN,

    LPAREN, RPAREN, LBRACE, RBRACE, LBRACKET, RBRACKET,
    COMMA, COLON, SEMICOLON, DOT, ARROW

    END_OF_FILE,
    INVALID
};
```

Np. dla:

```
int x = 123
```

```
Token{TokenType::KW_INT, "int", ...}  
Token{TokenType::IDENT, "x", ...}  
Token{TokenType::ASSIGN, "=", ...}  
Token{TokenType::INT_LITERAL, "123", ...}
```

Tablica symboli

```
enum class TypeKind {  
    Basic,  
    List,  
    UserDefined,  
    Void  
};  
  
enum class BasicType {  
    Int,  
    UInt,  
    Float,  
    Bool,  
    Char,  
    String  
};  
  
struct Type {  
    TypeKind kind;  
  
    BasicType basic; // gdy Basic  
    std::shared_ptr<Type> elementType; // gdy List  
    std::string typeName; // gdy UserDefined
```

```
};
```

Powyżej szkic kodu C++ przechowującego rodzaje typów w języku.
Dla `list<list<float>>` da nam to:

Type

```
+ kind = List
+ elementType -> Type
    + kind = List
    + elementType -> Type
        + kind = Basic
        + basic = Float
```

Jeśli scope zdefiniujemy w sposób zbliżony do poniższego, będziemy mogli przechodzić w górę, do rodziców Scope, w poszukiwaniu zmiennej podczas interpretacji.

```
struct Scope {
    std::string scopeName;
    Scope* parent;
    std::unordered_map<std::string, SymbolEntry> symbols;
};
```

Można ogólnie opisać sposób przechowywania w tablicy symboli jak poniżej:

GLOBAL SCOPE

```
|— zmienne
|— funkcje
|— klasy
```

CLASS INFO

```
|— pola
|— metody
```

FUNCTION INFO

```
|— parametry
|— typ zwracany
|— ciało AST
```

LOCAL SCOPE

```
|— parametry
|— zmienne lokalne
```

W takim przypadku wewnątrz Scope (jako słownik `string : SymbolEntry`) przechowywać zmienne (o danym Type), klasy i funkcje. Wtedy pola i metody klas są częścią obiektu czegoś jak `ClassInfo`, parametry i zmienne lokalne funkcji są zapisane wewnątrz obiektu `FunctionInfo`. Można oddzielnie przechowywać informację o istnieniu danej zmiennej/klasy/funkcji oraz jej wartości podczas runtime.

```
Scope { map<string, Symbol>, parent }
RuntimeScope { map<string, Value>, parent }
```

e lokalne funkcji są zapisane wewnątrz obiektu FunctionInfo. Można oddzielnie przechowywać informację o istnieniu danej zmiennej/klasz/funkcji oraz jej wartości podczas runtime.

Jeśli chodzi o metody klasy, to mogą one być z lub bez kwalifikatora mut, a informację o tym można przechowywać wewnątrz ClassInfo

```
class User {
    name: string
    age: int

    fun printName() -> void { ... }
    fun isAdult() -> bool { ... }
    mut fun setName(name: string) -> void { ... }
    mut fun birthday() -> void { ... }
}
```

CLASS: User

Pola

nazwa	typ
name	string
age	int

Metody

nazwa	parametry	zwraca	mut?
printName	()	void	nie
isAdult	()	bool	nie
setName	(name: string)	void	tak
birthday	()	void	tak

W tablicy symboli można by także przechowywać zapewne namespace, wtedy dla:

```
// file2.djm
fun fun2() -> void { ... }
class User { ... }
int x = 5 // globalna stała
```

```
// main.djm
import fun2 from "file2.djm"
```

Moglibyśmy podczas analizy semantycznej file2.djm utworzyć poniższą tablicę symboli

MODULE SCOPE: file2

nazwa	rodzaj	info
fun2	function	() -> void
User	class	definicja klasy
x	variable	int, global const

A następnie zapisać ją wewnątrz Scope pliku main.djm - i podczas wykonywania programu main.djm gdy widzimy użycie file2.fun2() możemy spojrzeć do tablicy symboli, znaleźć tam wpis pod std::string file2, i wtedy przejść do tablicy symboli (inny scope) utworzonej dla pliku file2.djm i tam odnaleźć wpis pod w słowniku pod std::string fun2.

SCOPE: main module

nazwa	rodzaj	info
file2	namespace	wskazuje na scope modułu file2

Parser

Może rozważmy działanie parsera na przykładzie. Załóżmy poniższy plik .djm

```
import * from "math.djm"
```

```
class User {
    name: string
    age: int
}
```

```

    fun isAdult() -> bool {
        return age >= 18
    }
}

fun greet(u: User) -> string {
    return u.name
}

int MAGIC = 5

```

Ogólnie parser mógłby go widzieć jako:

MODULE AST: main.djm

nr	rodzaj elementu	zawartość
1	import	import * from "math.djm"
2	class declaration	class User { ... }
3	function decl	fun greet(u: User) -> string ...
4	global variable	int MAGIC = 5

Co rozkłada na poniższe ASTNode

IMPORT NODE

pole	wartość
import kind	wildcard
imported file	"math.djm"
alias/module	"math"
selected names	brak

CLASS DECLARATION NODE

pole	wartość
class name	User
fields	[name:string, age:int]
methods	[isAdult, setName]

FIELDS IN CLASS User		
nr	nazwa	typ

1	name		string	
2	age		int	
+-----+				
METHODS IN CLASS User				
+-----+				
nr	nazwa		mut?	
nr	parametry		zwraca	
+-----+				
1	isAdult		nie	
1	()		bool	
2	setName		tak	
2	(newName:string)		void	
+-----+				

Oczywiście ciało metody greet wewnątrz klasy User jest przechowywane jako blok AST, nie jako tekst

FUNCTION DECLARATION NODE

+-----+		+-----+	
pole		wartość	
+-----+		+-----+	
function name		greet	
parameters		[u:User]	
return type		string	
body		block node	
+-----+		+-----+	

Dalej mamy deklarację zmiennej (właściwie globalnej stałej) MAGIC

VARIABLE DECLARATION NODE

+-----+		+-----+	
pole		wartość	
+-----+		+-----+	
name		a	
declared type		int	
mutable		tak	
initializer		literal 5	
+-----+		+-----+	

Parser oczywiście jedynie zapisuje strukturę tego przypisania, a dopiero analizator semantyczny sprawdzi jego poprawność (tu jest dobrze, ale moglibyśmy mieć przykładowo `int MAGIC = "abc"`).

Sposób, w jaki parser zapisuje deklarację funkcji wraz z jej ciałem mógłby wyglądać jak poniżej:

```
fun add(x: int, y: int) -> int {
    int z = x + y
    return z
}
```

FUNCTION NODE: add

Nagłówek:

+-----+		
nazwa	add	
parametry	x:int, y:int	
zwraca	int	
+-----+		

Body = BLOCK

1. variable declaration: `int z = x + y`
2. return statement: `z`

EXPRESSION NODE

+-----+		
rodzaj	binary expression	
+-----+		
operator	+	
left operand	identifier x	
right operand	identifier y	
+-----+		

Jeśli chodzi o jakieś przykładowe wywoływania metody, mogłyby funkcjonować w następujący sposób:

```
u.setName("Anna")
```

METHOD CALL EXPRESSION

+-----+		
receiver	identifier u	
method name	setName	
arguments	["Anna"]	
+-----+		

```
file2.fun2()
```

QUALIFIED CALL EXPRESSION

+-----+		
left	identifier file2	
right	identifier fun2	
arguments	()	
+-----+		

Oczywiście parser nie zna typu `u`, nie wie czy klasa której `u` jest obiektem ma w ogóle metodę `setName`, czy `u` jest mutowalne i czy potencjalnie metoda `setName` ma kwalifikator `mut`.

Podobnie nie wie także, czy `file2` to namespace czy obiekt utworzonej przez użytkownika klasy, oraz czy `fun2` istnieje w tym namespace. Wszystko to ustala dopiero analizator semantyczny.

Analizator semantyczny

Analizator semantyczny dostaje od parsera AST modułu (pliku .djm), to znaczy pewną listę wszystkich deklaracji i wyrażeń. Początkowo przechodzi on po najwyższym poziomie i tworzy odpowiednie wpisy w tablicy symboli

importy	-> namespace
klasy	-> symbole klas
funkcje	-> symbole funkcji
globalne stałe	-> symbole zmiennych globalnych

Następnie wchodzi do ciał funkcji, i wewnątrz nich tworzy lokalne scope'y, umieszcza parametry jako zmienne w lokalnym scope danej funkcji, sprawdza typy wyrażeń i rozwiązuje nazwy (przeszukując utworzoną dotychczas tablicę symboli). Na tym poziomie analizator semantyczny może zgłaszać błędy semantyczne do modułu C++ odpowiedzialnego za rejestrowanie błędów.

A więc krok po kroku dla prostego pliku .djm:

```
class User {  
    name: string  
    age: int  
}  
  
fun greet(u: User) -> string {  
    string s = u.name  
    return s  
}
```

Krok 1: Parser buduje AST

MODULE AST

1. **ClassDecl**(User)
2. **FunctionDecl**(greet)

ClassDecl(User)

```
fields:  
  - name : string  
  - age  : int  
methods:  
  - brak
```

FunctionDecl(greet)

```

params:
  - u : User
return type:
  - string
body:
  1. VarDecl(s : string = u.name)
  2. Return(s)

```

Krok 2: Analizator tworzy globalny scope

SCOPE: global

nazwa	rodzaj	info
User	class	definicja klasy
greet	function	(User)->string

Krok 3: Analizator analizuje klasę i tworzy ClassInfo User

CLASS INFO: User

Pola

nazwa	typ
name	string
age	int

Metody

(brak)

Krok 4: Analizator analizuje funkcję greet

SCOPE: greet

parent -> global

nazwa	rodzaj	typ
u	parameter	User
s	variable	string

Przy string `s = u.name` najpierw znajduje typ `u` (który może być lokalną zmienną wewnątrz scope funkcji, niekoniecznie oddzielny rodzaj parameter jak powyżej), dowiadyuje się że typ `u` to `User`, sprawdza czy `User` ma pole `name` (a wcześniej utworzył `ClassInfo User` i tam można to sprawdzić) oraz jakiego typu jest `name` i czy można go przypisać do string `s` (w projektowanym języku właściwie nie ma automatycznej konwersji zmiennych przy przypisywaniu, więc `name` musi mieć po prostu typ `string`).

Przed taką analizą semantyczną parser mógł utworzyć przy okazji string `s = u.name` poniższy `ASTNode`

```
MemberAccess(receiver=u, member=name)
```

A po analizie semantycznej analizator semantyczny może uzupełnić go o poniższe informacje

```
MemberAccess(receiver=u, member=name)
resolved receiver type = User
resolved member = field User.name
expression type = string
```

Co może pomóc interpreterowi, ułatwić jego implementację i przyspieszyć jego działanie.

Zgłaszanie błędów

Z racji, że błędy mogą występować zarówno w warstwie leksykalnej (nieprawidłowy token, np. nazwa zmiennej zaczynająca się od cyfry), przy parsowaniu (rozłożenie tokenów niezgodne z gramatyką) jak i przy okazji analizy semantycznej, konieczne jest wprowadzenie zunifikowanego mechanizmu zgłaszania błędów. Przewiduję konieczność wprowadzenia klasy (jak `ErrorManager`), w której zapisywane będą wewnątrz jakiejś kolekcji zgłaszane błędy (obiekty klasy `Error`) - będą one miały wewnątrz zapisane której warstwy (lekser, parser, analizator semantyczny) dotyczą błędy, ich wagę (`error`, `warning`) oraz linię i kolumnę w której błąd wystąpił (co jest ułatwione przez trzymanie wewnątrz Tokenu właśnie tej linii i kolumny).

Dzięki takiemu podejściu można zwyczajnie zbierać błędy w trakcie wykonywania programu, i kontynuować jego wykonanie, jeśli uznamy, że dany błąd jest "dopuszczalny" (np. brakujący nawias klamrowy otwierający po `if`ie, może warto spróbować go dostawić i zobaczyć co się stanie? Temat poruszany podczas wykładu).

Przykłady błędów

```
fun add(x: int, y: int -> int {
    return x + y
}
```

Zakładam, że projektowany język (jego interpreter) będzie dokonywał próby naprawy powyższego kodu poprzez wstawienie sztucznie nawiasu zamykającego - jest to prosty do rozpoznania przykład. Zgłosi przy tym poniższy błąd wraz z notatką:

```
Error [Syntax] at line 1, column 24: expected ')' after parameter list, got '->'
Note: assuming missing ')' before '->'
```

W przypadku niepowodzenia (dla innych przykładów) naprawy automatycznej programu zakładam wypisanie błędów - oczywiście pierwszy błąd spowoduje kaskadę następnych, co jest częstym problemem kompilatorów np. C++. Kaskada błędów dla powyższego kodu (z zapomnianym nawiasem) mogłaby wyglądać tak:

```
Error [Syntax]: expected ')'
Error [Syntax]: unexpected '->'
Error [Syntax]: unexpected 'int'
Error [Syntax]: expected '{'
Error [Syntax]: unexpected 'return'
```

Jest mniej czytelna dla programisty, który musiałby tak naprawdę naprawić pierwszy błąd i uruchomić program ponownie, aby zorientować się ile spośród wyświetlanych błędów (w dalszej części programu mogłyby też być inne błędy) jest spowodowanych przez kaskadę.

```
int 5abc = 2
Error [Lexical] main.djm:3:5: invalid identifier '5abc': identifiers must start with a letter
```

```
int a = 5
string s = "abc"
int x = a + s
Error [Semantic]: cannot apply operator '+' to types (int, string)
list<float> xs = [2.5, "a"]
Error [Semantic]: expected element of type float, got string
```

```
list<float> xs = [2.5, 5, 3.1]
Error [Semantic]: cannot assign int to float without explicit conversion (use 'as float')
```

```
int a = 5
a = 7
Error [Semantic]: cannot assign to immutable variable 'a'
```

```
User u = User{...}
u.setName("Anna")
Error [Semantic]: cannot call mut method 'setName' on immutable object 'u'
```

Gramatyka EBNF

program = { import_decl } , { top_level_decl } ;

// Dyskusyjne czy import powinien być wymagany na początku
// przez gramatykę, ale może nie jest to okropne ostatecznie

```
top_level_decl = class_decl
                | function_decl
                | global_const_decl ;     // Może tu variable_decl_stmt?
```

```
import_decl    = "import" , import_spec , "from" , string_literal ;
```

```
import_spec    = "*"
                | identifier , { "," , identifier } ;
```

```
global_const_decl
                = type , identifier_list , [ "=" , expression ] ;
//        Być może niekonieczne
```

```
class_decl     = "class" , identifier , "{" , { class_member } , "}" ;
```

```
class_member   = field_decl
                | method_decl ;
```

```
field_decl     = identifier_list , ":" , type ;
```

```
method_decl    = [ "mut" ] , "fun" , identifier ,
                "(" , [ parameter_list ] , ")" ,
                "->" , type ,
                block ;
```

```
function_decl   = "fun" , identifier ,
                "(" , [ parameter_list ] , ")" ,
                "->" , type ,
                block ;
```

```
parameter_list = parameter , { "," , parameter } ;
```

```
parameter      = identifier , ":" , type ;
```

```
type           = basic_type
                | list_type
                | user_type ;
```

```
basic_type     = "int"
                | "uint"
                | "float"
                | "bool"
                | "char"
                | "string"
```

```

        | "void" ;

list_type      = "list" , "<" , type , ">" ;

user_type      = identifier ;

block          = "{" , { statement } , "}" ;

statement      = variable_decl_stmt
                | assignment_stmt
                | expr_stmt
                | return_stmt
                | if_stmt
                | while_stmt
                | for_stmt
                | block ;

variable_decl_stmt
    = [ "mut" ] , type , identifier_list , [ "=" , expression ] ;

assignment_stmt    = assignment_target , "=" , expression ;

assignment_target
    = postfix_expr ;

expr_stmt          = expression ;
return_stmt        = "return" , [ expression ] ;

if_stmt            = "if" , expression , block ,
                    { "else" , "if" , expression , block } ,
                    [ "else" , block ] ;

while_stmt         = "while" , expression , block ;

for_stmt           = "for" , type , identifier , "in" , expression , block ;

expression         = reduce_expr ;

reduce_expr        = contains_expr ,
                    [ "reduce" , "with" , expression , "from" , expression ] ;

contains_expr      = logical_or_expr ,
                    { "contains" , logical_or_expr } ;

logical_or_expr    = logical_and_expr ,
                    { "||" , logical_and_expr } ;

logical_and_expr

```



```

    = equality_expr ,
      { "&&" , equality_expr } ;

equality_expr = relational_expr ,
  { ( "==" | "!=" ) , relational_expr } ;

relational_expr = additive_expr ,
  { ( "<" | "<=" | ">" | ">=" ) , additive_expr } ;

additive_expr = multiplicative_expr ,
  { ( "+" | "-" ) , multiplicative_expr } ;

multiplicative_expr =      pipeline_expr , { ( "*" | "/" | "%" ) , pipeline_expr } ;

pipeline_expr = filter_expr , { ">" , filter_expr } ;

filter_expr = cast_expr , { "?" , cast_expr } ;

cast_expr = power_expr , { "as" , type } ;

power_expr = unary_expr , [ "^" , power_expr ] ;

unary_expr = unary_op , unary_expr
            | postfix_expr ;

unary_op    = "-"
            | "!"
            | "count"
            | "reverse"
            | "flatten" ;

postfix_expr = primary_expr , { postfix_op } ;

postfix_op   =      "." , identifier , [ call_suffix ]
              | index_suffix
              | slice_suffix
              | call_suffix ;

call_suffix  = "(" , [ argument_list ] , ")" ;

index_suffix = "[" , expression , "]" ;

slice_suffix = "[" , [ expression ] , ":" , [ expression ] , "]" ;

argument_list = expression , { "," , expression } ;

primary_expr = literal
            | identifier

```

```

| list_literal
| object_literal
| "(" , expression , ")"
| "this" ;
    // Użycie this poza ciałem metody klasy jest błędem semantycznym, co
    // nie znaczy, że to już parser musi się temu sprzeciwiać

```

```
list_literal  = "[" , [ expression , { "," , expression } ] , "]" ;
```

```
object_literal = identifier , "{",
    [ field_initializer , { "," , field_initializer } ] ,
    "}";
```

```
field_initializer
    = identifier , ":" , expression ;
```

```
literal      = int_literal
    | float_literal
    | string_literal
    | char_literal
    | bool_literal ;
```

```
bool_literal  = "true" | "false" ;
```

```
identifier_list = identifier , { "," , identifier } ;
```

```
identifier    = letter , { letter | digit | "_" } ;
```

```
int_literal   = "0"
    | nonzero_digit , { digit } , { "" , digit , { digit } } ;
```

```
float_literal = digit , { digit } , "." , digit , { digit } ;
```

```
string_literal = "\"" , { string_char } , "\"" ;
```

```
char_literal  = "'" , char_char , "'" ;
```

```
letter        = "a" | "b" | "c" | "d" | "e" | "f" | "g" | "h" | "i" | "j"
    | "k" | "l" | "m" | "n" | "o" | "p" | "q" | "r" | "s" | "t"
    | "u" | "v" | "w" | "x" | "y" | "z"
    | "A" | "B" | "C" | "D" | "E" | "F" | "G" | "H" | "I" | "J"
    | "K" | "L" | "M" | "N" | "O" | "P" | "Q" | "R" | "S" | "T"
    | "U" | "V" | "W" | "X" | "Y" | "Z" ;
```

```
digit         = "0" | nonzero_digit ;
nonzero_digit = "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9" ;
```

```
string_char  =    // dowolny znak oprócz końca linii;  
char_char    =    // to samo?
```